

가능성을 제품으로 만들고, 끝까지 책임지는 Product Engineer 김대정입니다.

기획자로 시작해 백엔드로 왔고, 지금은 아이디어를 제안한 AI 콘텐츠 제품을 고객이 구독하는 서비스로 만들고 운영합니다. 커머스·피부와 운영·SNS 콘텐츠 세 도메인에서 현업의 반복 업무를 제품으로 만들었고, AI 활용이 본격화되면서 모두가 메이커로 제품을 만들고 운영하는 팀이라, 백엔드 경험이 적은 팀원도 같은 기준으로 만들 수 있는 백엔드 기준을 직접 세웠습니다.

JYP에서 먼저 확인할 세 가지

01 Coding Agent를 위한 Backend 기본값

백엔드 경험이 적은 팀원도 Coding Agent와 같은 구조로 운영 제품을 만들 수 있게 FastAPI 기준을 template로 만들었습니다.

02 고객이 구독하는 AI 제품

자료 정리·초안·검수·예약·발행을 하나의 흐름으로 만들고 실제 고객에게 운영했습니다.

03 대화형 제품 인터페이스

여러 제품 기능을 대화 하나로 제어하고 변경 작업은 사용자가 확인하는 방식을 검증했습니다.

팀 개발 체계

01

백엔드 경험이 적은 팀원도 Coding Agent로 운영 제품의 기능을 만들 수 있게 했습니다

기능 구현을 대신하는 방식이 아니라, 반복해서 문제가 생기던 session-transaction 경계를 FastAPI Template의 기본값으로 만들었습니다.

한 일	백엔드 경험이 적은 담당자도 Claude Code·Codex로 기능을 구현할 수 있도록 FastAPI-SQLAlchemy async 기반 Backend Template을 직접 설계·구축했습니다. 신규 사내 프로그램은 이 Template으로 시작하고, 기존 제품은 DB session 관리부터 순차적으로 적용했습니다.
문제	Local에서 정상처럼 보이던 코드가 STG QA에서 session 미반납으로 connection pool을 고갈시키고 500 오류를 만들었습니다. Backend 담당자가 매번 원인을 찾고 코드를 보완해야 했습니다.
해결의 핵심	Service가 @transactional로 transaction policy를 선언하고 Repository가 ContextVar의 현재 AsyncSession을 사용하도록 했습니다. Session-transaction-API contract는 공통값으로 두고, 일반 기능은 MVC, AI Agent 기능은 Hexagonal 구조를 선택하도록 기준을 정했습니다.
결과	Full Template으로 시작한 신규 프로그램의 STG QA에서는 같은 session-pool 문제가 다시 관측되지 않았습니다. Backend의 개입도 기능 구현 지원에서 결과 피드백과 배포 지원 중심으로 줄었습니다.

제품 개발·의사결정 업무 분장 — 판단은 사람이, 맥락 찾기와 추천은 agent가

업무 체계 · 역할 · 게이트

역할 구조

역할 코드로 전결을 지칭. 역할 → 사람 해소는 조직 서비스 조회

경영 결정 층

회사 방향 · 계약과 제도 · 기술과 장애 체계 · 판매 방식을 축별로 확정. 축마다 전결권자가 다르다

본부장 · PO

제품 방향 · 우선순위 · 일정 · UI/UX-STG 게이트 컨펌 (기획 리더 겸임)

제품 스쿼드 × N

역할·책임은 같고 사람 배치만 다르다

Tech Lead

구현 · 아키텍처 책임 · WBS · 우선순위 · 피드백 조율

기획

기능 · 정책 · UX 상세 확정

디자인

화면 표현 최종 (전 스쿼드 겸임)

개발

자기 태스크 스펙 작성 · 구현 (리더 합의 범위)

QA

제품 단위 배치 · 검증 전략 · 판정 · 이슈 등록

챗터 · 지원

기준 수립 · 난제 (실행 지원)

백엔드 챗터 리더

Tech Lead 겸임 · 챗터 기준 수립

프론트 챗터 리더 · QA 리더

QA 리소스 배분 · 검증 전략 컨펌

R&D · 경영지원

기술 문의 상위선 · 외부 창구 · CS-VOC 수신

제품 실행 흐름 — 문서가 곧 실행 원장

Decision → SPEC → Work Package → QA → release

제품 단위

Decision

범위 · 정책 · 벤더 · 비용 · 일정 결정 기록

계약으로

SPEC

resource · status · enum · API 계약만 환류

1 WP = 1 파일

Work Package

Owner lane: Backend · Frontend · QA · Ops

QA case 실행

Release Gate

Scope · Code · Spec · UX · QA · Approval

version cut

release note · trace

릴리스 노트 자동화 · 버전 추적

WP status — draft → in_dev → ready_for_qa → qa_blocked → ready_for_release — done

Release approval ----- Product · QA · Tech Lead · (범위·정책·비용·일정·고객 약속 변경 시) Decision Owner ----- release

의사결정 흐름 3분기 — 등록 안착과 수신자 (AI gate는 추천만, 확정은 사람)

흐름	보고자 → 수신자	위임전결표에서 뽑는 것	안착 상태 · 다음 사람 액션	수신자 실명화 근거	agent가 하는 일
결정 요청	요청자 → 결정자	그 사안 부분의 전결권자를 그대로 수신자로	Registered(결정 대기) → 결정자가 결정 → Decided	조직 서비스 조회 (역할 코드 → 사람)	맥락 찾기 · 실행안 정리 · 수신자 추천 사람)
실행 요청	요청자 → 수행자	전결권자 = 요청 정당성 체크 + 실행 라인 좁히기	Registered(실행 대기) → 수행자 착수 → Executing (결정 단계 스킵)	제품별 기획·개발 ·QA 담당 live read	동일
공유	보고자 → 검토자	전결권자 → 검토자. 「누가 시켰나」가 아니라 「이 사안 부분의 결재선」	Feedback에 직접 안착 → 결과 확인 → Done / Executing	조직 서비스 조회 (상급 라인)	동일. 명시 지목이 있으면 실재·활성만 확인

의사결정 원장 상태별 처리 - 발생 → 등록 → 결정 → 실행 → 피드백 → 종료를 하나의 원장으로

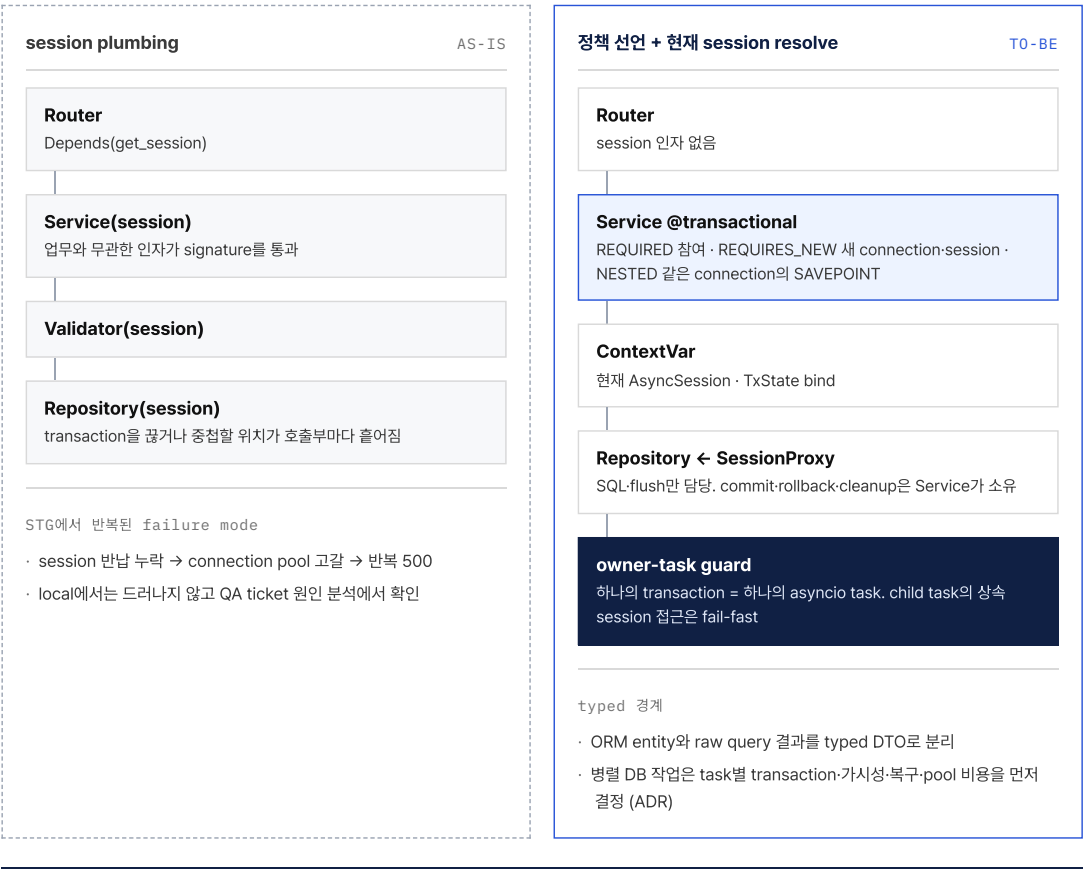
상태	진입 조건	이 상태에서 하는 일	성공 전이	실패 전이	관측·개입
Registered	품 제출로 등록. 원장·추적 시작점	흐름 유형에 따라 결정 대기 또는 실행 대기로 안착	Decided / Executing	Dropped (일반 업무·중복·기존 연결)	등록 전 Slack 탐색·추천·승인은 원장 상태가 아님
Decided	결정자 승인	파생 실행 태스크 생성	Executing	OnHold (보류 → 재검토 / 종료 대기)	결정자·결정 시각
Executing	실행 착수	파생 태스크 진행	Feedback	Blocked (블로커로 멈춤) → 해소 시 복귀	blocker·일정
Feedback	실행 결과 보고·공유 흐름은 여기서 직접 안착	결과 확인 루프	Done → Closed (회고까지)	Executing (추가 실행)	수신자 확인
Closed·Dropped	회고까지 마침 / 라이프사이클 없이 종료	append된 event로 이력 보존	없음 (terminal)	없음	모든 전이는 decision_event append

불변 조건	판단이 필요한 단계(우선순위·결정·배정·승인·release)는 사람이 확정한다. 역할 → 사람은 문서가 아니라 조직 서비스가 해소한다.
버린 대안	위임전결표 DB화(보류)·사람 명단을 문서에 이중으로 두는 것·단일 장문 문서 (agent read 상한에 뒷부분이 잘려 결정자 칸이 비던 사고 → 8,000자 미만 파일로 분할)
검증 근거	제품 단위 Decision·SPEC·WP·release gate 적용·운영 기록·release note 자동화·담당자가 바뀌어도 이어지는 인수인계 (수치 없음, 정성 표현까지만)

사람 이름·제품 수·담당 배치·전결 매트릭스는 적지 않는다. 짚은 박스가 본인 위치(Tech Lead·백엔드 챗터 리더)다. 전사 문서 pipeline 규칙 정의는 다른 저자가 주도했고, 본인은 제품 단위 Decision·SPEC·Work Package·release gate 적용·운영을 리드했으며 회사 업무 AX 구조 설계에 참여했다.

transaction-session 경계 — session 인자 전달을 없애고 Service가 정책을 선언

판단 비교 · AS-IS / TO-BE



직접 구축 · FastAPI Template-transaction/session 기준-architecture guide
운영 주도 · 제품별 결정→명세→작업→QA→릴리스 흐름
설계 참여 · 회사 업무 AX 확장 구조

기능 구현은 각 담당자와 Claude Code-Codex가 수행했고, 저는 결과 피드백과 인프라 배포를 지원했습니다. 관측 범위는 신규 프로그램의 STG QA입니다.

Threads 글쓰기의 반복을 고객이 구독하는 제품으로 만들었습니다

여러 채널을 함께 운영하느라 Threads를 꾸준히 관리하기 어려운 고객의 문제에서 출발했습니다. 참고 자료 수집부터 생성·검수·예약·발행까지 이어지는 흐름을 만들고, 팀과 함께 고객이 구독료를 내는 제품으로 출시·운영했습니다.

이 버전에서 보는 지점

사람이 반복하던 콘텐츠 제작 흐름을 재설계하고 FastAPI 백엔드·별도 AI 서비스를 직접 구현하고 주요 화면은 coding agent로 완성해 유료 제품으로 운영한 경험입니다.

사람이 하던 일

짧은 글 한 편에도 다섯 단계의 판단이 반복됐습니다

소재를 찾고 각도를 정하고, 초안을 쓰고, 발행한 뒤 반응을 다음 글에 연결하는 일을 한 사람이 계속 기억해야 했습니다.

01

소재 포착
글감 찾기



02

각도 잡기
첫 문장 정하기



03

초안 작성
내 톤으로 쓰기



04

검수·발행
최종 판단



05

반응 해석
다음 글에 반영

발행 반응을 다시 소재와 방향으로 연결해야 다음 글을 이어갈 수 있음

기억과 반복이 사람에게 남음

제품으로 바꾼 흐름

반복 업무는 역할과 데이터 흐름으로 나누고, 방향과 발행은 사람이 결정하게 했습니다

글쓰기 기능 하나를 붙인 것이 아니라 소재·정체성·작성·검수·학습을 나누고, 승인된 개인 맥락만 다음 생성에 다시 쓰는 흐름을 설계했습니다.

운영 제품예 구현 설계·단계 검증 사람이 결정

01 · CONTEXT

개인 맥락

설문 · 목표 · 내 콘텐츠



사람이 승인한 맥락

Identity · Evidence · Learning

02 · CREATE

Scout

소재 · 근거

Friend

관점 · 톤



Creator / Writer

초안 생성



Critic / Guard

품질 검수



Draft Feed

고를 수 있는 초안

03 · DECIDE

사람

고르기 · 수정 · 승인



예약 · 발행

승인된 결과만 반영



반응 · 수정

다음 생성에 쓸 변경안 제안

사용자 수정과 반응은 자동으로 장기 기억에 넣지 않고, 사람 검토를 거쳐 승인된 내용만 다음 생성에 다시 사용합니다.

구현과 운영

제품 판단부터 Backend·AI·주요 화면과 운영까지 이어 맡았습니다

FastAPI 제품 API, 별도 AI 애플리케이션과 DB, 콘텐츠 가져오기·URL preview·생성·예약·발행·대시보드·관리자 화면을 구현했습니다.

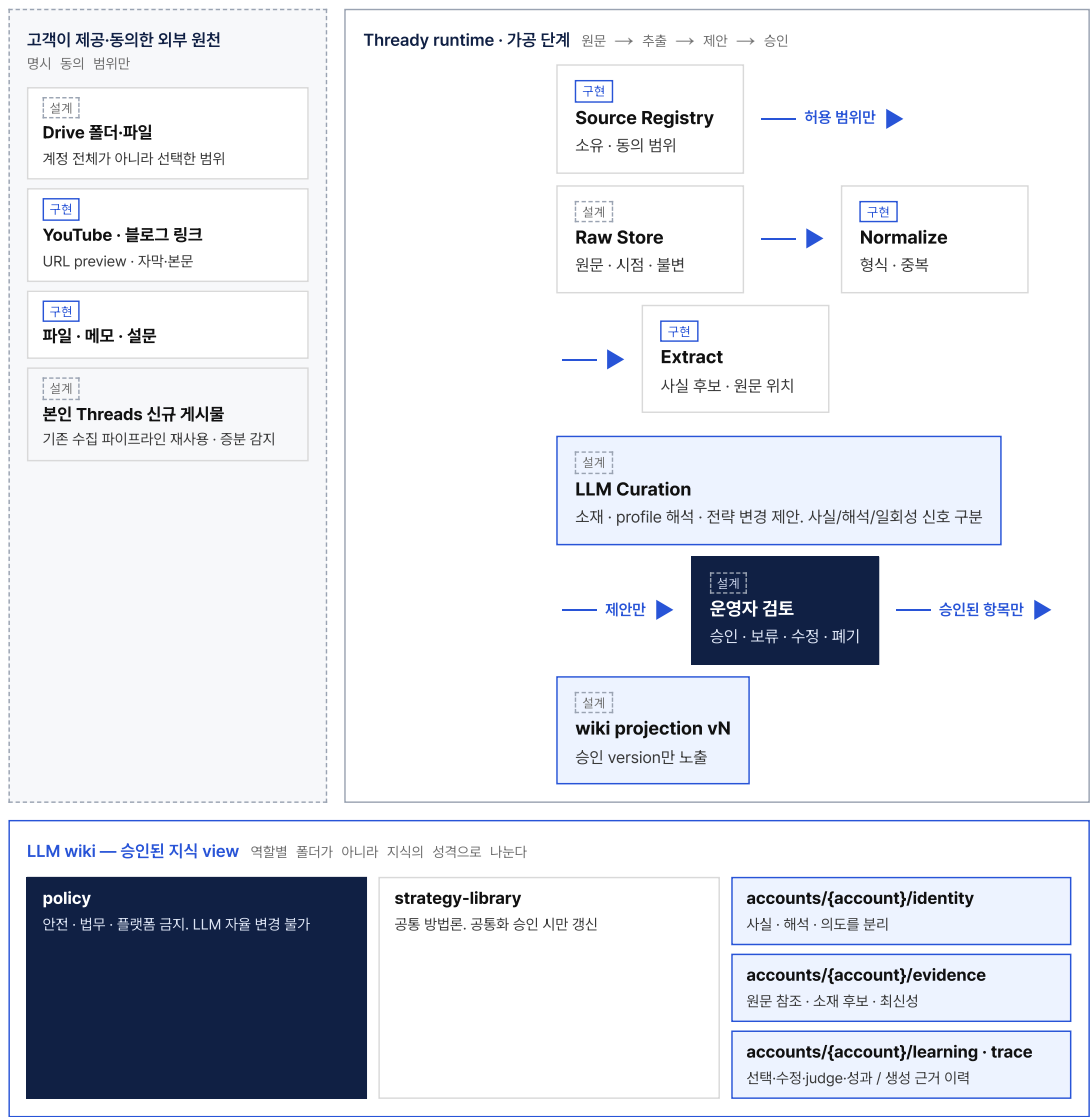
planner/writer가 초안을 만들고 LLM judge가 1차 검수 근거를 남기며, 사용자가 수정과 최종 발행을 결정하도록 구성했습니다.

외부 AI 5xx는 Sentry로 감지하고 반복 장애 모델을 일시 제외해 정상 모델로 작업을 이어갈 수 있게 운영했습니다.

설계

Thready 개인 계정 AX ① — raw 원문은 wiki에 바로 쓰지 않는다

AX 설계 · 지식 파이프라인



지식 항목 상태별 처리

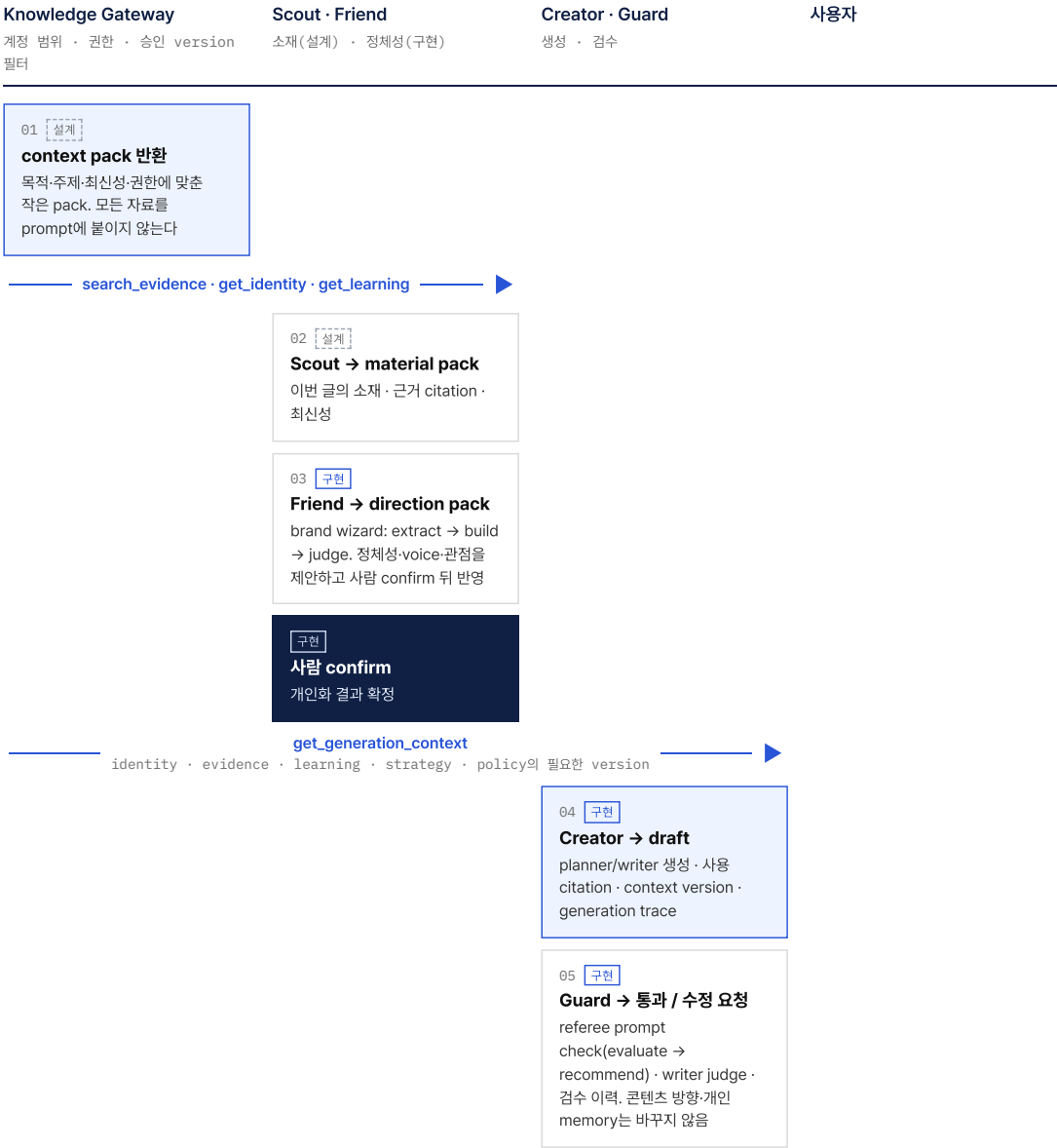
상태	진입 조건	이 상태에서 하는 일	다음 전이	철회·정정 시	관측·개입
raw	동의된 원천에서 수집	원문-시점-checksum 보관. 원칙적으로 불변	extract 생성	권한 철회 → source 비활성. downstream 재검토	마지막 수집 시점
extract	raw 존재	텍스트-사실 후보-원문 위치. 추출기 개선 시 재생성 가능	LLM curation 입력	해당 raw 기준으로 다시 생성	추출물마다 raw 참조
제안	LLM curation	변경 대상-근거-적용 범위 명시	운영자 승인 → projection	보류·수정·폐기	승인 대기 수
projection vN	승인	역할별 read view. 승인 version만 노출	agent가 gateway로 읽음	영향받는 항목만 되돌리거나 폐기	version별 사용 trace
trace	생성 완료	읽은 version-근거-결과-평가. append-only	learning change request → 검토로 복귀	원문 철회 시 접근 차단·영향 범위 표시	회고·재현

불변 조건	raw 원문은 wiki에 바로 쓰지 않는다. wiki에는 승인된 version만 올라간다. policy는 LLM이 바꾸지 않는다.
버린 대안	모든 자료를 prompt에 붙이는 방식· Drive를 유일한 Raw Store로 두고 생성 시마다 외부 원천을 읽는 방식· 원문-추출-해석을 한 저장소에 쉬는 방식
검증 근거	설계안(exploration) + 운영 코드의 콘텐츠 가져오기-URL preview-source ownership 검증. 파일럿 최소 범위는 운영자 등록 → 추출 → 제안 → 승인 → 초안-trace → learning 후보

「구현」은 운영 Thready 코드에서 확인된 요소, 「설계」는 설계 문서 단계다. 이 설계안은 exploration이며 제품 계약이 아니다. 설계와 콘텐츠 가져오기-생성-검수
workflow 구현은 직접 담당했다.

Thready 개인 계정 AX ㉔ — agent는 gateway를 통해서만 승인된 지식을 읽는다

AX 설계 · 역할 그래프



학습 루프

설계

Threads 발행 → 반응

선택·수정·반응이 학습 데이터

judge · 성과

설계

learning change request

trace의 평가 결과. wiki를 직접
덮지 않는다

앞단 운영자 검토로 복귀

설계

승인분만 다음 projection

장기 memory 자동 승격 없음

역할별 계약 — 읽는 것 · 만드는 것 · 직접 바꾸지 않는 것

역할	읽는 정보 · tool	만드는 결과	다음 역할로	직접 바꾸지 않는 것	구현 상태
설계 Scout	search_evidence — 주제· 최신성·계정 범위의 승인 evidence	material pack (소재 · citation · 최신성)	Friend · Creator 입력	raw-extract 처리, identity-strategy-policy	자동 소재화 계약만 선언. 트리거추가·승인 흐름 미확정
구현 Friend	get_identity · get_learning · search_evidence	direction pack (profile · voice · 관점 · 의도)	사람 confirm → Creator	raw 원문, 공통 strategy, 장기 memory 직접 승격	brand wizard graph(extract → build → judge)로 구현. gateway형 tool 계약은 설계
구현 Creator	get_generation_context — 필요한 version만 묶은 context pack	draft · 사용 citation · generation trace	Guard	wiki 장기 지식, policy, 발행 승인	writer 역할 객체 + repair/judge 루프로 구현
구현 Guard	draft · 적용 policy · evidence citation	통과 / 수정 요청과 이유	사용자 선택	콘텐츠 방향 · 개인 memory	결정적 출구 검증 + writer judge + referee prompt check(evaluate → recommend → finalize)

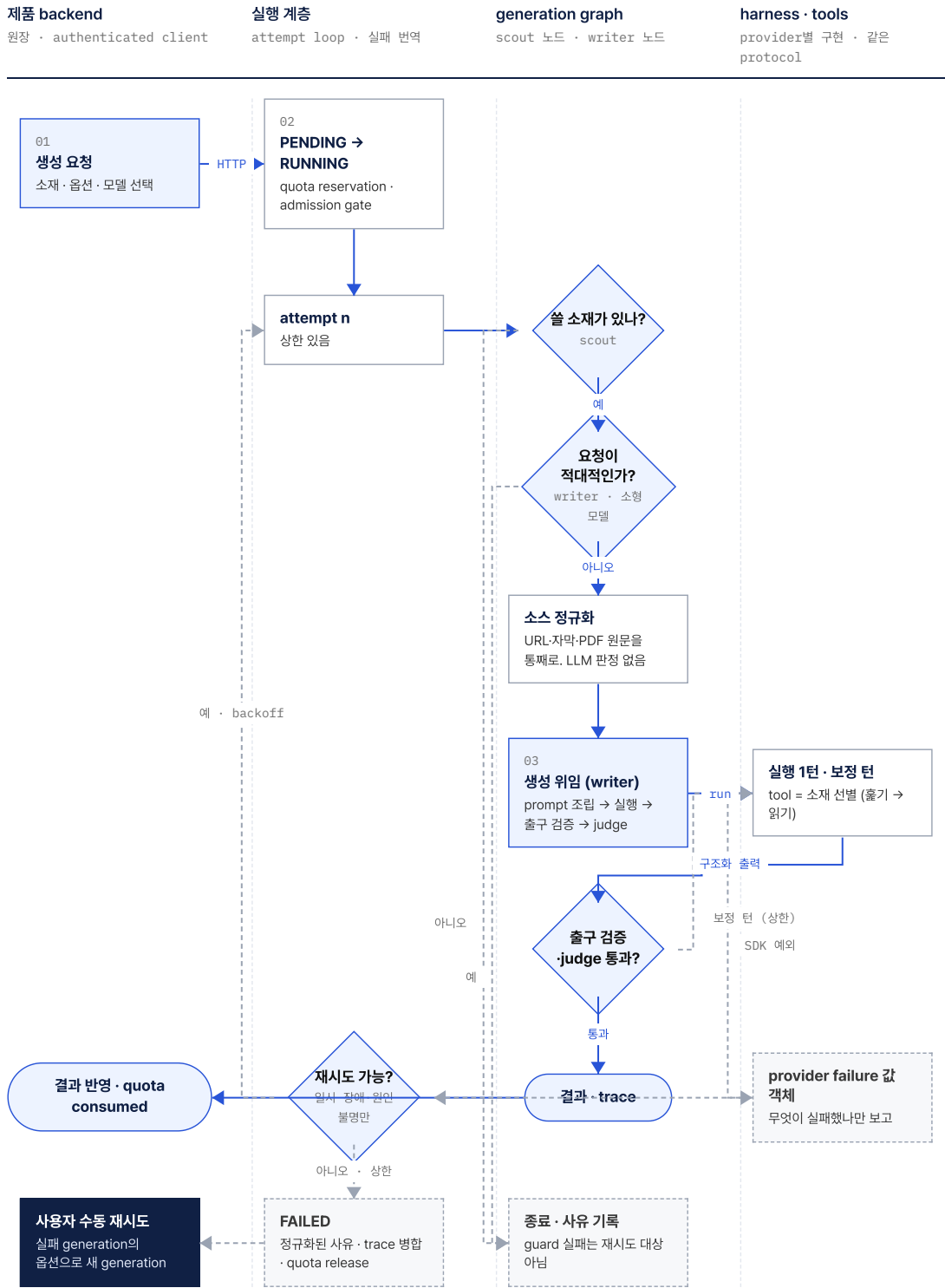
불변 조건	agent는 gateway가 돌려주는 승인 version만 읽는다. 평가 결과는 wiki를 직접 덮지 않고 앞단 검토로 되돌아간다. 발행 여부는 사용자가 정한다.
버린 대안	production Scouter를 먼저 만드는 것 (동의 범위·수집 주기·승인 흐름이 미확정) · 사람 확인 없는 자동 발행 · 장기 memory 자동 승격
검증 근거	설계안의 역할 계약 표 + 운영 코드의 friend(brand wizard) · writer · referee(prompt check) · scheduler(발행 시점) 역할 객체와 예약· 발행 화면. Scout의 production 구현은 주장하지 않는다

Scout는 설계 단계, Friend(brand wizard)-Creator(writer)-Guard(referee-judge)-발행 시점 제안(scheduler)-사용자 발행 선택은 운영 구현이 확인된 요소다.
사람 게이트는 세 곳: 운영자 승인, Friend 개인화 confirm, 사용자 발행 선택.



불변 조건	돌아가는 서비스를 멈추지 않는다. 재구축은 규칙(패턴·계층·하네스)을 먼저 세우고 그 위에서 한다.
버린 대안	부분 수정 유지 (재발 누적) · frontend까지 동시 교체 (회귀 범위 확대) · 규칙 없이 AI로 바로 생성
검증 근거	commit 실측 순서 · backend commit 대다수 본인 author · Jira 전이 전수 분석
관측 결과	QA 버그 재발률(해결 대비 reopen) 37% → 11%, 재발 발생 일평균 약 94% 감소. BE/FE 라벨이 없어 제품 전체 품질 지표로 서술

속도 우선으로 빠르게 검증된 초기 제품을 운영 단계로 전환한 판단이다. 전임 작업의 평화가 아니라 시점과 비용의 비교다.



generation 상태별 처리

상태	진입 조건	이 상태에서 하는 일	성공 전이	실패 전이	관측 · 개입
PENDING	backend 요청 수락. quota reserved	worker 실행 대기. admission gate 계산에 포함	worker 시작 → RUNNING	취소 → CANCELLED	대기 수 · 복구 surface
RUNNING	worker 시작	attempt loop. graph 4노드 · 보정 루프 · trace 기록	반환 가능한 결과 ≥ 1 → SUCCESS (quota consumed)	재시도 소진-비재시도 → FAILED / 사용자 취소 → CANCELLED	attempt · 노드별 trace
SUCCESS	결과 생성	종료. 결과-citation-trace 보존	없음 (terminal)	없음	quota charge 대상
FAILED	결과 0개	정규화된 failure reason (source · provider · guard · output). quota release	사용자 수동 재시도 (새 generation)	없음	reason별 집계 · 오류 추적

불변 조건	역할은 LLM 지능만 소유한다. 판정은 그 입력을 아는 역할 안에 둔다 (소재 유무 = scout, 요청 적대성 = writer). 재시도 정책은 실행 계층이 소유한다.
버린 대안	plan → generate → fact guard 3노드 + repair loop의 구 graph 엔진 (agent 엔진이 전 환경 게이트를 통과한 뒤 폐기) · 소스 map/reduce 압축과 LLM 유용성 분석 · 자동 circuit breaker-failover (당시 규모에서 운영 이점이 작다고 판단)
검증 근거	backend-AI application 전체 회귀 · 보정/judge 루프의 발행 계약 검증 · 5xx 분류·재시도·terminal failure 검증. friend-referee-scheduler 역할 객체도 같은 규칙으로 구현

역할은 LLM 지능만 소유하고 DB-transaction은 service가 소유한다. 재시도 여부는 AI 계층이 아니라 실행 계층이 정한다. provider 실명은 적지 않는다.

담당 범위

제품 운영을 리드하고 백엔드·AI·주요 프론트엔드 흐름을 직접 구현했습니다. 기획·QA·마케팅은 각 담당자와 함께 수행했으며, 제품 매출은 팀의 결과로 구분합니다. 외부 채널에서 소재를 자동으로 찾는 기능과 사람 확인 없는 자동 발행은 구현 범위로 주장하지 않습니다.

대화형 제품 인터페이스

03

Planner-Capability Registry-상태 원장 직접 구현

여러 화면의 기능을 대화 하나로 다루는 방식을 검증했습니다

사용자가 기능의 위치와 실행 순서를 외우는 대신, 원하는 작업을 말하면 Agent가 필요한 Thready 기능을 선택하고 결과를 대화로 이어서 보여주는 독립 프로토타입을 만들었습니다.

이 버전에서 보는 지점

사용자가 기능의 위치와 순서를 외우는 대신 원하는 작업을 말하면 Agent가 등록된 Thready 기능을 선택하는 흐름을 구현했습니다. 외부 상태 변경은 Mock gateway에서만 검증했습니다.

사용 흐름 비교

기능을 찾아다니던 사용 흐름을 대화 하나로 모았습니다

화면을 하나 더 만드는 실험이 아니라, 이미 동작하는 Thready 기능을 사용자의 의도에 맞춰 고르고 이어서 실행하는 인터페이스를 검증했습니다.

사용자가 기능과 순서를 직접 찾아 실행

AS-IS

화면 01

계정

작업 계정 선택

화면 02

소재

글감 찾기

화면 03

작성

기획·초안·수정

화면 04

게시물

저장·예약발행

화면 05

대시보드

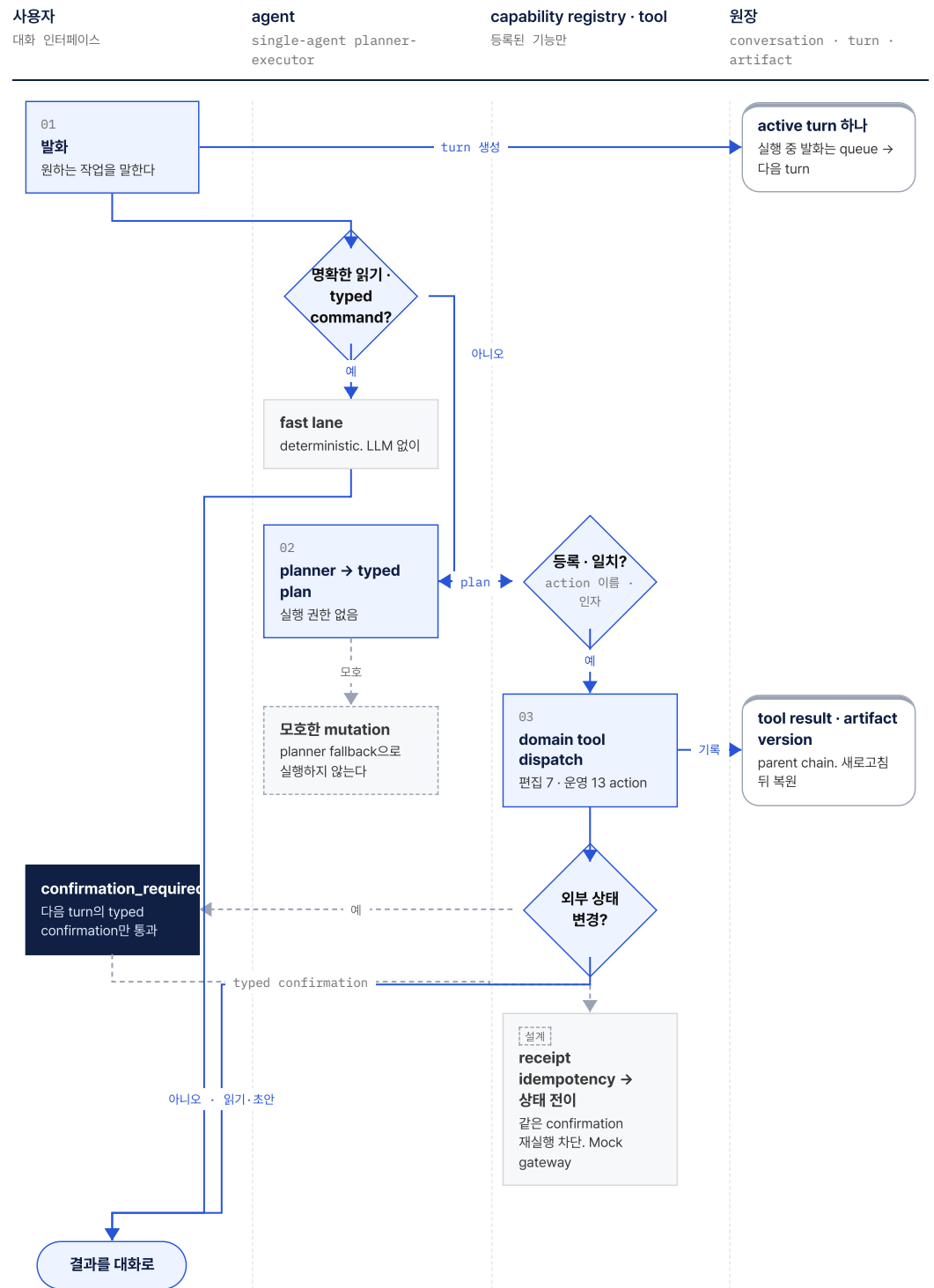
성과 확인

기능이 준비되어 있어도 사용자는 어디서 무엇을 해야 하는지 알고, 화면을 옮길 때마다 현재 작업의 맥락을 직접 이어야 했습니다.

T0-BE · 설계

대화형 제품 제어 agent — planner는 계획만, 실행은 등록된 capability만, 상태 변경은 확인 뒤에만

흐름 · 실행 권한 분리



불변 조건	planner는 실행 권한을 갖지 않는다. 등록되지 않은 action은 실행되지 않는다. 외부 상태 변경은 typed confirmation과 receipt를 통과할 때만 일어난다.
버린 대안	multi-agent orchestration (범위 대비 복잡) · 최근 message 개수 절단 (token-aware context selection + append-only compaction snapshot으로 대체)
검증 근거	대화-queue-artifact · 편집 action · activity · capability · Mock operation 각 test 통과. 운영용 durable worker와 process restart recovery는 미구현 범위

독립 prototype이다. 편집 기능은 기존 writer를 재사용했고, 계정-게시물 운영 action은 Mock gateway에서 실행 흐름과 안전 경계만 검증했다. 실제 예약·발행·삭제는 수행하지 않았다.

작업의 위험도에 따라 실행 규칙을 나눴습니다

Agent가 계획을 만들더라도 모든 기능을 같은 방식으로 실행하지 않습니다. 조회, 결과물 생성, 외부 상태 변경에 서로 다른 경계를 적용했습니다.

조회

계정 · 게시물 · 성과

바로 실행하고 대화에 결과 반환

생성·수정

초안 · 편집 · 재작성

결과물 버전을 남긴 뒤 다음 요청과 연결

외부 상태 변경

예약 · 발행 · 삭제

사용자 확인과 중복 실행 방지를 거쳐 실행

계획, 실행, 결과를 다시 이어갈 수 있게 기록했습니다

콘텐츠 제작 7개, 제품 운영 13개 동작을 capability로 등록

계획 단계와 실제 기능 실행 권한을 분리

대화·turn·도구 결과·결과물 버전을 실행 기록에 적재

작업 큐와 activity event로 진행 상태를 대화에 복원

대화가 길어지면 이전 맥락을 요약해 다음 요청에 연결

확인 절차와 idempotency key로 중복 상태 변경 방지

기존 Writer와 소재 관리에서 검증한 기능을 재사용한 독립 프로토타입입니다. 실제 예약·발행·삭제는 Mock gateway까지만 연결했으며, 운영용 durable worker와 실제 외부 상태 변경은 구현 범위에 포함하지 않았습니다.